

Assembleia IA - Especificacao Completa v3.0

ASSEMBLEIA IA

ESPECIFICAÇÃO TÉCNICA COMPLETA E PROMPT MESTRE EXECUTÁVEL

Product Requirements Document + Architecture + Database + APIs + Agents + Security + Backlog

PARTE 0 — PROMPT MESTRE ABSOLUTO (COPIE E COLE NO AGENTE)

PARTE 1 — VISÃO DE PRODUTO E NEGÓCIO

- 1.1 Declaração de Visão
- 1.2 Problema que Resolve
- 1.3 Solução
- 1.4 Personas Principais
- 1.5 Modelo de Negócio (SaaS)
- 1.6 Métricas de Sucesso (North Star)

PARTE 2 — ARQUITETURA DE SISTEMA

- 2.1 Princípios Arquiteturais
- 2.2 Diagrama Lógico de Alto Nível (texto)
- 2.3 Estrutura do Monorepo (Turborepo)
- 2.4 Stack Tecnológica Detalhada e Justificada
 - Frontend Web
 - Mobile
 - Backend Principal
 - Camada de IA
 - Dados
 - Infra

PARTE 3 — MODELO DE DADOS COMPLETO (SCHEMA)

- 3.1 Princípios do Schema
- 3.2 Tabelas Core (lista completa)
 - Tenancy e Auth
 - Pessoas e CRM
 - Frequência e Presença
 - Eventos
 - Escalas
 - Financeiro
 - Patrimônio
 - Conteúdo e Sermões
 - Comunicação
 - Website / CMS
 - Agentes e IA
 - Auditoria e LGPD
 - Marketplace
 - Billing (SaaS)
- 3.3 Exemplo de RLS (PostgreSQL)
- 3.4 Índices Críticos

PARTE 4 — AUTENTICAÇÃO, AUTORIZAÇÃO E MULTI-TENANT

- 4.1 Fluxo de Login
- 4.2 Matriz de Permissões (RBAC + ABAC)
- 4.3 Isolamento de Dados

PARTE 5 — AGENTES DE IA (ESPECIFICAÇÃO COMPLETA)

- 5.1 Arquitetura de Agentes
- 5.2 Agente Mestre — System Prompt Base
- 5.3 Agentes Especializados — Capacidades e Tools
 - Agente Pastoral
 - Agente Financeiro
 - Agente Secretaria
 - Agente Escalas
 - Agente Designer IA
 - Agente Vídeo IA

Agente Sermões + RAG
Agente Analytics
Agente Jurídico / LGPD
Agente Comunicação + Social Media
Agente RH Ministerial + Eventos

5.4 Human-in-the-Loop

PARTE 6 — WHATSAPP COMO CENTRO DE OPERAÇÕES

- 6.1 Arquitetura
- 6.2 Exemplos de Comandos e Respostas Esperadas
- 6.3 Segurança no Canal

PARTE 7 — PORTAIS E EXPERIÊNCIA DO USUÁRIO

- 7.1 Portal Administrativo (Web)
- 7.2 Portal do Membro
- 7.3 Website Institucional por Igreja
- 7.4 Aplicativo Mobile

PARTE 8 — ESTÚDIOS DE MÍDIA

- 8.1 Estúdio Criativo (Imagem)
- 8.2 Editor de Vídeo IA

PARTE 9 — FINANCEIRO E PATRIMONIAL (DETALHES)

- 9.1 Fluxos Financeiros Críticos
- 9.2 Patrimônio

PARTE 10 — SEGURANÇA, PRIVACIDADE E CONFORMIDADE

- 10.1 Controles Técnicos
- 10.2 LGPD
- 10.3 Auditoria

PARTE 11 — APIs PÚBLICAS E WEBHOOKS

- 11.1 Estilo
- 11.2 Principais Grupos de Endpoints
- 11.3 Webhooks de Saída

PARTE 12 — OBSERVABILIDADE E OPERAÇÕES

PARTE 13 — TESTES E QUALIDADE

PARTE 14 — DEPLOYMENT E AMBIENTES

PARTE 15 — MARKETPLACE DE AGENTES E EXTENSÕES

PARTE 16 — INTERNACIONALIZAÇÃO E ACESSIBILIDADE

PARTE 17 — BACKLOG DETALHADO POR FASES

- FASE 0 — Fundação (2-3 semanas)
- FASE 1 — Core + Agente Mestre + WhatsApp (3-4 semanas)
- FASE 2 — Operações Essenciais (4 semanas)
- FASE 3 — Mídia e Comunicação (4 semanas)
- FASE 4 — Pastoral e Avançado (3-4 semanas)
- FASE 5 — Escala, Marketplace e Polish (contínuo)

PARTE 18 — SCRIPTS DE MCP (LINHA DE COMANDO)

PARTE 19 — REQUISITOS NÃO FUNCIONAIS (EXPANDIDOS)

PARTE 20 — RISCOS E MITIGAÇÕES

PARTE 21 — GLOSSÁRIO

PARTE 22 — INSTRUÇÕES FINAIS AO AGENTE DE DESENVOLVIMENTO

APÊNDICE A — SCHEMA SQL COMPLETO (DDL)

APÊNDICE B — ENDPOINTS REST (CATÁLOGO)

APÊNDICE C — USER STORIES ESSENCIAIS

APÊNDICE D — SYSTEM PROMPTS

APÊNDICE E — STATE MACHINES

APÊNDICE F — ERROS PADRÃO

APÊNDICE G — GO-LIVE CHECKLIST

APÊNDICE H — MODELOS IA

APÊNDICE I — CUSTO IA

APÊNDICE J — ESCALA

APÊNDICE K — install-mcps.sh

APÊNDICE L — FASES (LEMBRETE)

1. CONTEXTO DENOMINACIONAL

2. HIERARQUIA ESTRUTURAL TÍPICA (IGREJA LOCAL AD)

Níveis territoriais suportados no sistema

3. CATÁLOGO COMPLETO DE CARGOS E FUNÇÕES DA ASSEMBLEIA DE DEUS

- 3.1 Cargos Eclesiásticos / Ministeriais (consagração)
 - Observações por convenção
- 3.2 Cargos de Direção de Unidade
- 3.3 Cargos Administrativos da Igreja Local
- 3.4 Escola Bíblica Dominical (EBD)

- 3.5 Círculo de Oração / Senhoras / Mulheres
- 3.6 Departamento de Varões / Homens
- 3.7 Mocidade / Jovens / Adolescentes / Crianças
- 3.8 Louvor, Música e Artes
- 3.9 Mídia, Comunicação e Tecnologia
- 3.10 Recepção, Ordem e Apoio ao Culto
- 3.11 Intercessão, Visitação e Ação Social
- 3.12 Ensino, Discipulado e Cursos
- 3.13 Outros cargos frequentes
- 4. DEPARTAMENTOS PADRÃO (SEED AD)
- 5. MODELO DE DADOS — CARGOS, VÍNCULOS E HISTÓRICO
 - 5.1 Tabelas novas / expandidas
 - 5.2 Campos adicionais em `people` (membro)
 - 5.3 Regras de negócio do vínculo cargo
 - 5.4 Permissões sugeridas por categoria de cargo
- 6. FLUXOS ESPECÍFICOS AD
 - 6.1 Admissão de membro
 - 6.2 Consagração de obreiro
 - 6.3 Transferência (carta de mudança)
 - 6.4 Disciplina
- 7. SEED SQL / TYPESCRIPT (RESUMO PARA O AGENTE)
- 8. EXEMPLOS DE COMANDOS EM LINGUAGEM NATURAL (WHATSAPP / CHAT)
- 9. RELATÓRIOS OBRIGATÓRIOS (AD)
- 10. CHECKLIST DE IMPLEMENTAÇÃO DESTE VOLUME

ASSEMBLEIA IA

ESPECIFICAÇÃO TÉCNICA COMPLETA E PROMPT MESTRE EXECUTÁVEL

Product Requirements Document + Architecture + Database + APIs + Agents + Security + Backlog

Versão: 2.0.0 — Enterprise Edition

Data: 29 de julho de 2026

Classificação: Documento Mestre Oficial — Fonte Única da Verdade

Público-alvo: Agentes de IA de desenvolvimento (Antigravity, Cursor, Claude Code, Devin, Aider, Windsurf) + Engenheiros humanos

Escopo: Sistema SaaS multi-tenant de gestão inteligente de igrejas com IA como núcleo

Meta de completude: Documento auto-contido. O agente deve implementar 100% sem perguntas adicionais.

PARTE 0 — PROMPT MESTRE ABSOLUTO (COPIE E COLE NO AGENTE)

Você é o Arquiteto-Chefe e Engenheiro Principal Full-Stack + Especialista em Sistemas Multi-Agente de IA + Especialista em SaaS Religioso Enterprise.

MISSÃO ÚNICA E IRREVOCÁVEL:

Construir do zero a plataforma **Assembleia IA** exatamente conforme este documento de especificação v2.0.0.

REGRAS INVIOVÁVEIS:

- Este documento é a ÚNICA fonte da verdade. Não invente requisitos fora dele, mas implemente TUDO o que está aqui.
- IA (agentes LangGraph + MCP) é o núcleo. Nenhuma feature crítica pode existir sem integração com agentes.
- Multi-tenant desde o commit zero (PostgreSQL RLS + tenant_id em tudo).
- WhatsApp Business Cloud API é o principal canal de operação humana.
- Stack obrigatória: Next.js 15 + NestJS + FastAPI + LangGraph + PostgreSQL 16 + pgvector + Redis + MinIO + RabbitMQ.
- Código 100% TypeScript (frontend + NestJS) e Python 3.12+ (agentes). Tipagem estrita.
- Monorepo Turborepo.
- Toda ação crítica tem Human-in-the-Loop configurável.
- LGPD completa + auditoria imutável.
- Gere: código, migrations, seeds, testes, Docker, K8s manifests, CI/CD, scripts MCP, documentação e prompts de sistema de cada agente.
- Prioridade de implementação: Fase 0 → Fase 1 → Fase 2... conforme backlog.
- Ao final de cada módulo, atualize o README e o CHANGELOG.
- Nunca deixe endpoints sem autenticação e autorização (RBAC + ABAC).
- Todo agente deve registrar suas ações na tabela de auditoria.
- Use Model Context Protocol (MCP) para TODAS as ferramentas externas e internas.

Comece imediatamente pela Fase 0 (Fundação). Crie a estrutura do monorepo, o schema completo do banco com RLS, o sistema de auth multi-tenant e o skeleton do Agente Mestre.

Você tem autorização total para criar qualquer arquivo, pasta, script ou configuração necessária.

PARTE 1 — VISÃO DE PRODUTO E NEGÓCIO

1.1 Declaração de Visão

O Assembleia IA será o **Sistema Operacional Inteligente** para igrejas de qualquer tamanho no mundo. Não é um ERP tradicional com chatbots colados. É uma plataforma onde **agentes autônomos de IA** executam a maior parte das operações administrativas, financeiras, comunicacionais, de mídia e pastorais, enquanto humanos aprovam decisões críticas e focam no cuidado de pessoas.

1.2 Problema que Resolve

Igrejas (especialmente de médio e grande porte) sofrem com: - Trabalho administrativo excessivo de secretárias e líderes - Falta de padronização de processos - Comunicação fragmentada (WhatsApp pessoal, planilhas, sistemas legados) - Dificuldade de acompanhar membros que deixam de frequentar - Geração de artes e vídeos cara e lenta - Prestação de contas financeira complexa e sujeita a erros - Escalas de voluntários manuais e injustas - Falta de inteligência sobre o que está acontecendo na igreja

1.3 Solução

Uma plataforma única, multi-tenant, com: - Agente Mestre orquestrador - 13+ agentes especializados - WhatsApp como interface principal - Portais web + app mobile - Estúdios de criação de arte e vídeo com IA - Financeiro e patrimonial completos - Conformidade LGPD nativa

1.4 Personas Principais

Persona	Descrição	Necessidades principais
Pastor Sênior	Líder espiritual da igreja	Visão geral, triagem pastoral, menos interrupções administrativas
Secretária	Operacional do dia a dia	Cadastros, documentos, escalas, atendimento
Tesoureiro	Responsável financeiro	Conciliação, relatórios, prestação de contas, PIX
Líder de Ministério	Coordena voluntários	Escala, comunicação, presença
Voluntário	Serve em departamentos	Ver escala, confirmar disponibilidade, receber avisos
Membro	Frequenta a igreja	Atualizar dados, contribuir, se inscrever, assistir cultos
Super Admin (SaaS)	Equipe Assembleia IA	Gerenciar tenants, billing, suporte, marketplace

1.5 Modelo de Negócio (SaaS)

Planos:

Plano	Membros	Preço base (BRL/mês)	Agentes incluídos	Limites principais
Essencial	até 200	R\$ 197	Mestre + Secretaria + Comunicação	5 usuários admin, 10 GB storage

Plano	Membros	Preço base (BRL/mês)	Agentes inclusos	Limites principais
Crescer	até 1.000	R\$ 497	+ Financeiro + Escalas + Analytics	15 usuários, 50 GB
Impactar	até 5.000	R\$ 1.297	+ Pastoral + Designer + Eventos + Social Media	40 usuários, 200 GB
Dominar	ilimitado	R\$ 2.997+	Todos + Vídeo IA + RH + Jurídico + prioridade	Ilimitado + SLA 99.95%

Add-ons: créditos extras de geração de imagem/vídeo, usuários adicionais, storage, white-label, API rate higher, agentes customizados do marketplace.

Billing: Stripe / Asaas / Mercado Pago. Trial 14 dias. Cancelamento a qualquer momento.

1.6 Métricas de Sucesso (North Star)

- % de tarefas administrativas executadas por agentes sem intervenção humana
- Tempo médio de resposta a solicitações via WhatsApp
- Taxa de retenção de membros (membros que voltam a frequentar após alerta da IA)
- NPS de pastores e secretárias
- MRR e churn mensal

PARTE 2 — ARQUITETURA DE SISTEMA

2.1 Princípios Arquiteturais

1. **Multi-tenant first** — isolamento por RLS + tenant_id
2. **Event-driven** — RabbitMQ/Kafka para comunicação entre serviços
3. **Agent-centric** — toda capacidade de negócio é exposta como tool MCP
4. **API-first** — tudo acessível via REST/gRPC; UI e WhatsApp são clientes
5. **Observable by default** — OpenTelemetry em todos os serviços
6. **Security by design** — zero trust interno, least privilege
7. **Idempotency** — operações financeiras e de publicação são idempotentes
8. **Human-in-the-loop** — ações irreversíveis exigem aprovação configurável

2.2 Diagrama Lógico de Alto Nível (texto)

```
[Usuários] → WhatsApp | Web | Mobile | API
↓
[API Gateway / Traefik] → Auth (NestJS) → Rate Limit → Tenant Resolver
↓
[NestJS Core API] ↔ [PostgreSQL + RLS] ↔ [Redis]
↓
[events]
[RabbitMQ / Kafka]
↓
[FastAPI AI Workers] ← LangGraph Agents ← MCP Servers
↓
[MinIO/S3] [OpenSearch] [External APIs: Meta, Google, YouTube, Gateways]
```

2.3 Estrutura do Monorepo (Turborepo)

```
assembleia-ia/
├── apps/
│   ├── web/                # Next.js 15 – Admin + Membro + Site builder
│   ├── mobile/             # Expo (React Native)
│   ├── api/                # NestJS – API principal, auth, webhooks
│   ├── ai-gateway/         # FastAPI – orquestração de agentes
│   ├── worker-video/       # Worker de processamento de vídeo (FFmpeg + IA)
│   ├── worker-media/       # Geração de imagens e artes
│   ├── landing/            # Site de marketing assembleiaia.com.br
│   └── docs/               # Documentação técnica (Nextra ou similar)
├── packages/
│   ├── database/           # Prisma ou Drizzle – schema, migrations, seeds
│   ├── ui/                 # Shadcn UI + design system
│   ├── auth/               # Biblioteca compartilhada de auth
│   ├── agents-core/        # Tipos, base classes, registry de agentes
│   ├── mcp-servers/        # Todos os servidores MCP
│   │   ├── whatsapp/
│   │   ├── financeiro/
│   │   ├── designer/
│   │   ├── video/
│   │   ├── membros/
│   │   ├── escalas/
│   │   ├── calendar/
│   │   ├── email/
│   │   ├── storage/
│   │   ├── analytics/
│   │   ├── lgpd/
│   │   └── social/
│   ├── config/             # ESLint, Prettier, TSConfig, Tailwind
│   ├── types/              # Tipos TypeScript compartilhados
│   └── utils/              # Helpers
├── infra/
│   ├── terraform/          # AWS/GCP/Azure
│   ├── k8s/                # Manifests Helm ou Kustomize
│   ├── docker/             # Dockerfiles
│   └── monitoring/          # Prometheus, Grafana dashboards
├── scripts/
│   ├── install-mcps.sh
│   ├── seed-demo-church.ts
│   ├── migrate.sh
│   └── backup.sh
├── tests/
│   ├── e2e/                # Playwright
│   ├── integration/
│   └── load/                # k6
├── .github/workflows/
├── turbo.json
├── package.json
└── README.md
```

2.4 Stack Tecnológica Detalhada e Justificada

Frontend Web

- **Next.js 15** (App Router, Server Components, Server Actions, Partial Prerendering)
- **React 19**
- **TypeScript 5.5+** (strict)
- **Tailwind CSS 4** + **Shadcn UI** + **Radix primitives**
- **Framer Motion** (animações)
- **TanStack Query v5** (cache e sincronização)
- **React Hook Form** + **Zod** (formulários e validação)
- **TipTap** (editor rich text para CMS e comunicados)
- **Fabric.js** ou **Konva.js** (canvas do Estúdio Criativo)

- **Apache ECharts** (dashboards)
- **next-intl** (i18n)
- **nuqs** (state na URL)

Mobile

- **Expo SDK 52+** (React Native) — preferencial por velocidade de desenvolvimento e OTA
- Alternativa: Flutter se performance nativa extrema for exigida

Backend Principal

- **NestJS 10/11** — modular, decorators, DI, guards, interceptors, OpenAPI automático
- **Prisma** ou **Drizzle ORM** (preferência Prisma por maturidade de migrations e type-safety)
- **Passport.js** + estratégias JWT, OAuth, local
- **BullMQ** ou **NestJS Bull** (filas internas leves)

Camada de IA

- **FastAPI** (Python 3.12) — alta performance para workers
- **LangGraph** — orquestração de multi-agentes com estado
- **LangChain** — utilitários quando necessário
- **Model Context Protocol (MCP)** — padrão de tools
- **pgvector** — embeddings e RAG
- Modelos: GPT-5.5 / Claude 4 / Gemini 2.5 (configurável por tenant) + modelos open-source locais quando privacidade exigir
- **Whisper** (transcrição) + **TTS** (voz)
- **FFmpeg** + bibliotecas de visão para vídeo

Dados

- **PostgreSQL 16** + extensão **pgvector** + **pg_partman** (particionamento de logs/auditoria)
- **Redis 7** (cache, sessões, rate limit, pub/sub, BullMQ)
- **MinIO** (S3-compatible) ou AWS S3
- **OpenSearch** (busca full-text de sermões, documentos, logs)
- **RabbitMQ** (preferencial para complexidade média) ou **Apache Kafka** (alta escala)

Infra

- Docker + Docker Compose (dev e staging)
- Kubernetes (produção)
- Traefik ou NGINX Ingress
- Terraform
- GitHub Actions
- Prometheus + Grafana + Loki + Tempo (OpenTelemetry)

- Cloudflare (DNS, CDN, WAF, R2 opcional)
-

PARTE 3 — MODELO DE DADOS COMPLETO (SCHEMA)

3.1 Princípios do Schema

- Toda tabela de negócio tem `tenant_id` `UUID NOT NULL`
- RLS (Row Level Security) ativado em todas as tabelas de tenant
- Soft delete (`deleted_at`) onde fizer sentido
- `created_at`, `updated_at`, `created_by`, `updated_by` padrão
- UUIDv7 ou UUIDv4 para IDs
- Enums PostgreSQL para status e tipos
- Índices compostos (`tenant_id` + campos de busca frequentes)
- Particionamento por tempo em tabelas de auditoria e mensagens

3.2 Tabelas Core (lista completa)

Tenancy e Auth

- `tenants` — `id`, `name`, `slug`, `domain`, `plan`, `status`, `settings (jsonb)`, `branding (jsonb)`, `created_at...`
- `users` — `id`, `email`, `phone`, `password_hash`, `name`, `avatar_url`, `status`, `mfa_enabled`, `last_login...`
- `tenant_users` — `tenant_id`, `user_id`, `role`, `permissions (jsonb)`, `status`
- `roles` — `id`, `tenant_id (null = system)`, `name`, `description`
- `permissions` — `resource`, `action`
- `role_permissions`
- `sessions` — `refresh tokens`, `device info`
- `mfa_secrets`
- `oauth_accounts`
- `api_keys` — para integrações e marketplace

Pessoas e CRM

- `people` — `id`, `tenant_id`, `type (member|visitor|leader|child)`, `full_name`, `preferred_name`, `birth_date`, `gender`, `marital_status`, `cpf`, `rg`, `email`, `phones (jsonb)`, `address (jsonb)`, `photo_url`, `baptism_date`, `membership_date`, `status`, `tags (text[])`, `custom_fields (jsonb)`, `lgpd_consent_at`, `notes...`
- `families` — `id`, `tenant_id`, `name`, `head_id`
- `family_members` — `family_id`, `person_id`, `relationship`
- `departments` — `id`, `tenant_id`, `name`, `parent_id`, `leader_id`
- `department_members`
- `spiritual_gifts / competencies`

- `person_competencies`
- `visits` — pastoral visits
- `counseling_sessions`
- `prayer_requests`
- `discipleship_tracks` + `person_discipleship_progress`

Frequência e Presença

- `attendance_events` — culto, reunião, evento
- `attendance_records` — `person_id`, `event_id`, `check_in_at`, `method` (qr|manual|facial)
- `absence_alerts` — gerados pela IA

Eventos

- `events` — `id`, `tenant_id`, `title`, `description`, `type`, `starts_at`, `ends_at`, `location`, `capacity`, `registration_required`, `budget_id`, `status`, `cover_image`...
- `event_registrations`
- `event_tickets` / `event_checkins`
- `event_schedules` (programação interna)

Escalas

- `scale_templates` — templates de escala por departamento
- `scales` — `id`, `tenant_id`, `department_id`, `event_id` ou `date`, `status`
- `scale_slots` — `role`, `required_count`
- `scale_assignments` — `person_id`, `slot_id`, `status` (confirmed|declined|pending), `notified_at`
- `availability` — `person_id`, `date ranges`, `notes`

Financeiro

- `financial_accounts` — caixa, banco, carteira digital
- `cost_centers`
- `chart_of_accounts`
- `transactions` — `id`, `tenant_id`, `type` (income|expense|transfer), `amount`, `currency`, `date`, `description`, `category_id`, `cost_center_id`, `person_id` (doador), `payment_method`, `status`, `reconciled`, `external_id`, `receipt_url`...
- `transaction_splits` (rateio)
- `budgets` + `budget_lines`
- `campaigns` (campanhas de arrecadação)
- `pledges` (compromissos de dízimo/oferta)
- `reconciliation_batches`
- `receipts`

Patrimônio

- `assets` — `id`, `tenant_id`, `name`, `category`, `serial`, `purchase_date`, `value`, `location`, `status`, `qr_code`, `warranty_until`, `maintenance_notes`
- `asset_movements`
- `maintenance_orders`

Conteúdo e Sermões

- `sermons` — `id`, `tenant_id`, `title`, `preacher_id`, `preached_at`, `series_id`, `audio_url`, `video_url`, `transcript`, `summary`, `verses` (jsonb), `tags`, `embedding vector`
- `sermon_series`
- `studies / devotionals`
- `media_library` — arquivos genéricos com embeddings

Comunicação

- `message_templates`
- `campaigns_communication`
- `message_logs` — whatsapp, email, push, sms
- `conversations` — threads WhatsApp/chat
- `conversation_messages`

Website / CMS

- `sites` — `tenant_id`, `domain`, `theme`, `settings`
- `pages` — `slug`, `title`, `content` (jsonb TipTap), `status`, `seo`
- `posts / news`
- `menus`
- `forms` + `form_submissions`

Agentes e IA

- `agent_definitions` — `name`, `version`, `system_prompt`, `tools`, `config`
- `agent_runs` — `id`, `tenant_id`, `agent_name`, `input`, `output`, `status`, `tokens_used`, `cost`, `started_at`, `finished_at`, `human_approval_required`, `approved_by`
- `agent_memories` — embeddings + metadata por tenant/usuário
- `approval_requests` — human-in-the-loop

Auditoria e LGPD

- `audit_logs` — particionada por mês — `actor`, `action`, `resource`, `old_value`, `new_value`, `ip`, `user_agent`, `tenant_id`
- `consent_records` — `person_id`, `purpose`, `granted_at`, `revoked_at`, `version`
- `data_subject_requests` — acesso, correção, exclusão, portabilidade

Marketplace

- marketplace_extensions
- tenant_extensions
- extension_reviews

Billing (SaaS)

- subscriptions
- invoices
- usage_records (créditos de IA consumidos)

3.3 Exemplo de RLS (PostgreSQL)

```
ALTER TABLE people ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation_people ON people
  USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

-- Função para setar o tenant no início de cada request
CREATE OR REPLACE FUNCTION set_tenant(tenant uuid) RETURNS void AS $$
BEGIN
  PERFORM set_config('app.current_tenant_id', tenant::text, true);
END
$$ LANGUAGE plpgsql;
```

O NestJS deve chamar `set_tenant` em um interceptor global após validar o JWT.

3.4 Índices Críticos

- (tenant_id, full_name) em people (trigram para busca fuzzy)
- (tenant_id, email), (tenant_id, phone)
- (tenant_id, date) em transactions
- Vector index (HNSW ou IVFFlat) em sermons.embedding e agent_memories
- (tenant_id, status, starts_at) em events
- GIN em tags e custom_fields (jsonb)

PARTE 4 — AUTENTICAÇÃO, AUTORIZAÇÃO E MULTI-TENANT

4.1 Fluxo de Login

1. Usuário acessa igreja.assembleiaia.com.br ou app
2. Resolve tenant pelo subdomain / header X-Tenant-ID / JWT claim
3. Login com e-mail + senha ou OAuth (Google, Microsoft, Apple)
4. Se MFA habilitado → TOTP ou SMS
5. Emite Access Token (15 min) + Refresh Token (30 dias, rotativo)
6. Claims do JWT: sub, tenant_id, roles[], permissions[], person_id (se vinculado)

4.2 Matriz de Permissões (RBAC + ABAC)

Roles padrão do sistema: - platform_superadmin - tenant_owner - tenant_admin - pastor - secretary - treasurer - ministry_leader - volunteer - member - visitor (acesso muito limitado)

Cada role tem permissões no formato resource:action (ex: people:read, people:write, finance:reconcile, agents:approve).

ABAC: regras adicionais baseadas em atributos (só pode editar pessoas do seu departamento, só vê transações do seu centro de custo, etc.).

4.3 Isolamento de Dados

- RLS no banco
- Prefixos de pasta no MinIO: tenants/{tenant_id}/...
- Filas e caches com chave tenant:{id}:...
- Logs e métricas com label tenant_id
- Nenhum job de background pode processar dados de outro tenant

PARTE 5 — AGENTES DE IA (ESPECIFICAÇÃO COMPLETA)

5.1 Arquitetura de Agentes

- Cada agente é um **grafo LangGraph** com estado tipado
- O Agente Mestre recebe a intenção do usuário e decide qual(is) agente(s) invocar
- Tools são **MCP servers** (ou funções Python/TypeScript registradas)
- Memória: short-term (estado do grafo) + long-term (pgvector por tenant)
- Toda execução gera registro em agent_runs
- Ações de alto risco geram approval_requests

5.2 Agente Mestre — System Prompt Base

Você é o Agente Mestre da plataforma Assembleia IA.
Você conhece todo o sistema: membros, finanças, eventos, escalas, comunicação, mídia, site, permissões e identidade visual da igreja atual (tenant).

Sua função é:

1. Entender a intenção do usuário em linguagem natural (português brasileiro e outros idiomas).
2. Planejar a sequência de passos necessários.
3. Delegar para agentes especializados via tools.
4. Coordenar resultados e apresentar um resumo claro.
5. Solicitar aprovação humana quando a ação for irreversível, financeira, de publicação em massa ou de exclusão de dados.

Nunca invente dados. Sempre use as tools. Respeite as permissões do usuário atual.
Mantenha o tom respeitoso, pastoral e profissional, adequado ao contexto de uma igreja.

5.3 Agentes Especializados — Capacidades e Tools

Agente Pastoral

Tools MCP: `membros.search`, `membros.get_history`, `visitas.create`, `aconselhamento.create`, `oracao.create`, `encaminhar_pastor`, `detectar_crise` **Comportamentos especiais:** - Classificação de urgência (baixa / média / alta / crise) - Detecção de linguagem de risco (protocolo de segurança: nunca fornece métodos, sempre encaminha e notifica pastor) - Resumo automático de longas conversas - Sugestão de acompanhamento e conteúdo

Agente Financeiro

Tools: `financeiro.lancar`, `financeiro.consultar_saldo`, `financeiro.conciliar`, `financeiro.gerar_relatorio`, `financeiro.emitir_recibo`, `financeiro.prever_caixa`, `financeiro.detectar_anomalia` **Regras:** - Toda saída acima de threshold configurável exige aprovação - Conciliação tenta matching automático por valor + data + descrição - Relatórios em linguagem natural + gráficos

Agente Secretaria

Tools: `membros.criar`, `membros.atualizar`, `documentos.emitir`, `batismo.registrar`, `casamento.registrar`, `transferencia.registrar`, `frequencia.registrar`

Agente Escalas

Algoritmo de geração: 1. Carregar template do departamento 2. Buscar voluntários elegíveis (competência + disponibilidade + não férias) 3. Aplicar restrições (não escalar a mesma pessoa N semanas seguidas, equilíbrio de carga, preferências) 4. Otimizar (heurística ou ILP simples) 5. Gerar proposta → notificar → aguardar confirmações → fechar escala

Agente Designer IA

Pipeline: 1. Recebe briefing (texto ou referência) 2. Carrega branding do tenant (logo, cores, fontes, guidelines) 3. Gera prompt otimizado para o modelo de imagem 4. Gera N variações 5. Usuário escolhe ou pede ajustes 6. Exporta nos formatos necessários (PNG, PDF, MP4 estático, etc.) 7. Opcionalmente publica

Agente Vídeo IA

Pipeline de culto típico: 1. Upload do vídeo bruto (ou link de gravação) 2. Transcrição (Whisper) 3. Detecção de segmentos (silêncio, mudança de cena, palavras-chave) 4. Geração automática de: - Versão full editada (cortes de silêncio, correção de áudio) - 5-15 shorts/reels com legendas - Highlights - Thumbnail 5. Revisão humana opcional 6. Publicação multi-canal com um clique

Agente Sermões + RAG

- Ingestão: áudio/vídeo → transcrição → chunking → embedding → pgvector
- Indexação de referências bíblicas (parser de versículos)
- Busca híbrida (keyword + semântica)
- Geração de resumo, esboço, estudo bíblico, série sugerida

Agente Analytics

- Converte pergunta em SQL seguro (somente SELECT, com tenant_id forçado)
- Ou consulta índices OpenSearch
- Gera resposta + tabela + gráfico (ECharts config)
- Exemplos de perguntas suportadas listados na seção 5 do documento original

Agente Jurídico / LGPD

- Gestão de bases legais e consentimentos
- Atendimento a direitos do titular (acesso, correção, exclusão, portabilidade)
- Relatórios de conformidade
- Alertas de retenção expirada

Agente Comunicação + Social Media

- Criação de textos com tom da igreja
- Agendamento e publicação
- Resposta a comentários (com aprovação quando sensível)
- Análise de engajamento e sugestões

Agente RH Ministerial + Eventos

- Cadastro de dons e competências
- Sugestão de pessoas para funções
- Organização completa de eventos (já descrita no Agente Mestre)

5.4 Human-in-the-Loop

Configurável por tenant e por tipo de ação: - Publicar em redes sociais
- Enviar mensagem em massa (> N destinatários) - Lançar despesa acima de valor X - Excluir pessoa ou dados - Alterar escala já publicada
- Qualquer ação marcada como `requires_approval: true` no tool

Fluxo: agente cria `approval_request` → notifica aprovadores (WhatsApp + painel) → timeout configurável → executa ou cancela.

PARTE 6 — WHATSAPP COMO CENTRO DE OPERAÇÕES

6.1 Arquitetura

- Número oficial da igreja (WABA) vinculado ao tenant
- Webhook Meta → NestJS → valida assinatura → enfileira → AI Gateway interpreta
- Sessão de conversa por `wa_id` + `tenant_id`
- Suporte a:
 - Texto
 - Áudio (transcreve)
 - Imagem
 - Documento
 - Localização
 - Botões e listas interativas
 - Templates aprovados (para mensagens proativas)

6.2 Exemplos de Comandos e Respostas Esperadas

Comando do usuário	Ação do sistema
"Cadastre o visitante João Silva, 11999999999"	Cria person type=visitor, confirma dados, oferece criar família
"Quem está escalado no louvor domingo?"	Consulta escala, responde lista formatada
"Quanto entrou de dízimo esta semana?"	Soma transactions, responde valor + comparativo
"Faça um banner para o culto de domingo"	Chama Designer, gera opções, envia imagens no WhatsApp
"Organize um retiro de casais em março"	Inicia fluxo do Agente Mestre (evento completo)
"Envie lembrete para os inscritos no congresso"	Lista inscritos, envia template, reporta quantos enviados

6.3 Segurança no Canal

- Apenas números autorizados (mapeados a users/people com permissão) podem executar ações administrativas
- Membros comuns só consultam seus próprios dados e fazem pedidos de oração/inscrição

- Rate limit por número
 - Todas as mensagens logadas
-

PARTE 7 — PORTAIS E EXPERIÊNCIA DO USUÁRIO

7.1 Portal Administrativo (Web)

Layout: Sidebar + Topbar + Conteúdo **Módulos do menu:** - Visão Geral (dashboard) - Pessoas (membros, visitantes, famílias) - Ministérios e Departamentos - Escalas - Eventos - Financeiro - Patrimônio - Comunicação - Mídia e Conteúdo - Site - Agentes de IA (histórico, aprovações, configurações) - Relatórios - Configurações (igreja, usuários, integrações, branding, LGPD)

Dashboard: cards de KPIs + gráficos + feed de atividades dos agentes + pendências de aprovação + agenda do dia.

7.2 Portal do Membro

- Home com próximos eventos, escala pessoal, pedidos de oração
- Meu perfil e família
- Contribuições e comprovantes
- Inscrições
- Conteúdo (sermões, estudos)
- Solicitar atendimento
- Carteirinha digital (QR Code)

7.3 Website Institucional por Igreja

- Editor visual (blocos)
- Temas e branding automático
- Páginas: Home, Sobre, Ministérios, Eventos, Sermões, Contato, Doação
- Formulários inteligentes
- Integração com streaming
- SEO e performance (Core Web Vitals)

7.4 Aplicativo Mobile

Telas principais: - Home - Agenda / Escala - Contribuição (PIX copia-cola + cartão) - Mídia - Chat / Pedidos - Perfil / Carteirinha - Notificações

Offline-first para carteirinha e conteúdos baixados.

PARTE 8 — ESTÚDIOS DE MÍDIA

8.1 Estúdio Criativo (Imagem)

- Canvas Fabric.js/Konva
- Camadas, textos, formas, imagens
- Geração por prompt com branding automático
- Remoção de fundo, upscale, restauração
- Templates da igreja + marketplace
- Exportação multi-formato
- Histórico de versões

8.2 Editor de Vídeo IA

- Timeline
 - Importação de cultos longos
 - Detecção automática de cortes
 - Legendas editáveis (Whisper)
 - Narração TTS / clonagem de voz (com consentimento)
 - Templates de intro/outro da igreja
 - Botão Publicar com checkboxes de canais
 - Fila de processamento (worker-video) com status em tempo real
-

PARTE 9 — FINANCEIRO E PATRIMONIAL (DETALHES)

9.1 Fluxos Financeiros Críticos

1. Recebimento de dízimo/oferta via PIX

- Gateway gera QR / copia-cola
- Webhook de confirmação
- Cria transaction automaticamente
- Emite recibo (PDF + WhatsApp)
- Atualiza dashboard

2. Conciliação bancária

- Importa OFX / Open Finance / CSV
- Matching automático (valor + data ± 2 dias + fuzzy descrição)
- Usuário resolve divergências
- Fecha lote

3. Prestação de contas mensal

- Agente gera balancete + DRE + gráficos
- PDF + apresentação
- Envia para conselho / assembleia

9.2 Patrimônio

- Cadastro com foto e QR Code
 - Empréstimo de equipamentos (check-out / check-in)
 - Ordens de manutenção
 - Inventário periódico (app mobile com leitura de QR)
 - Depreciação automática (opcional)
-

PARTE 10 — SEGURANÇA, PRIVACIDADE E CONFORMIDADE

10.1 Controles Técnicos

- TLS 1.3 em trânsito
- Criptografia em repouso (discos + campos sensíveis)
- Secrets em vault (HashiCorp Vault ou cloud secret manager)
- MFA obrigatório para roles administrativas
- Rotação de chaves
- WAF + rate limiting + bot protection
- Backup diário + PITR
- Testes de penetração periódicos

10.2 LGPD

- Base legal registrada por finalidade
- Consentimento granular
- Direitos do titular (portal + processo interno)
- DPIA para features de alto risco
- DPO (ou equivalente) configurável
- Retenção automática e exclusão
- Registro de atividades de tratamento

10.3 Auditoria

- Toda mutação de dados de negócio gera audit_log
 - Logs de autenticação e de agentes
 - Imutabilidade (append-only + assinatura)
 - Retenção configurável (mínimo 5 anos para dados financeiros)
-

PARTE 11 — APIs PÚBLICAS E WEBHOOKS

11.1 Estilo

- REST (OpenAPI 3.1 gerado automaticamente pelo NestJS)

- Versionamento por URL (/v1/)
- Autenticação: Bearer JWT ou API Key
- Paginação cursor-based
- Filtros e ordenação padronizados
- Idempotency-Key header em POSTs financeiros

11.2 Principais Grupos de Endpoints

- /auth/*
- /people/*
- /families/*
- /events/*
- /scales/*
- /finance/transactions/*
- /finance/reports/*
- /media/*
- /agents/run
- /agents/approvals/*
- /webhooks/whatsapp
- /webhooks/payments
- /sites/*
- /admin/tenants/* (apenas platform)

11.3 Webhooks de Saída

Tenant pode cadastrar endpoints para receber: - person.created / updated - transaction.confirmed - event.registration - scale.published - agent.run.completed

Assinatura HMAC-SHA256.

PARTE 12 — OBSERVABILIDADE E OPERAÇÕES

- OpenTelemetry traces em todos os serviços
 - Métricas: latência, taxa de erro, tokens de IA consumidos, custo estimado, filas
 - Logs estruturados (JSON) → Loki
 - Alertas: erro rate, fila parada, custo de IA anormal, falha de webhook Meta, disco, etc.
 - Runbooks para incidentes comuns
 - Status page pública
-

PARTE 13 — TESTES E QUALIDADE

- Unit tests (Jest / Vitest / Pytest) — cobertura > 80% no core
 - Integration tests (banco real em container)
 - E2E (Playwright) dos fluxos críticos
 - Contract tests das APIs
 - Load tests (k6) antes de cada release maior
 - Testes de segurança (SAST, DAST, dependency scanning)
 - Testes dos agentes com datasets de exemplos e avaliação de qualidade (LLM-as-judge quando aplicável)
-

PARTE 14 — DEPLOYMENT E AMBIENTES

Ambiente	Propósito	Dados
local	desenvolvimento	seed
staging	QA e demos	anonymized
production	clientes	reais

- Blue/green ou canary no Kubernetes
 - Migrations automáticas com lock
 - Feature flags (Unleash ou similar)
 - Rollback em < 5 minutos
-

PARTE 15 — MARKETPLACE DE AGENTES E EXTENSÕES

- Desenvolvedores terceiros podem publicar agentes e integrações
 - Review process + sandbox
 - Monetização (revenue share)
 - Instalação com um clique por tenant
 - Versionamento e permissões declaradas
-

PARTE 16 — INTERNACIONALIZAÇÃO E ACESSIBILIDADE

- i18n completo (pt-BR, en, es, no mínimo)
 - Datas, moedas, formatos locais
 - Suporte a múltiplas traduções bíblicas
 - WCAG 2.2 AA
 - Suporte a leitores de tela
 - Alto contraste opcional
-

PARTE 17 — BACKLOG DETALHADO POR FASES

FASE 0 — Fundação (2-3 semanas)

- ☐ Monorepo Turborepo + packages base
- ☐ Schema Prisma/Drizzle completo + RLS policies
- ☐ NestJS auth multi-tenant (JWT + MFA + OAuth)
- ☐ Interceptor de tenant + set_config
- ☐ Docker Compose (postgres, redis, minio, rabbitmq, mailhog)
- ☐ CI GitHub Actions (lint, test, build)
- ☐ Design system Shadcn básico
- ☐ Skeleton do Portal Admin (layout + auth)
- ☐ Seed de igreja demo

FASE 1 — Core + Agente Mestre + WhatsApp (3-4 semanas)

- ☐ CRUD de people + families
- ☐ FastAPI + LangGraph skeleton
- ☐ Agente Mestre com tools básicas (membros, agenda)
- ☐ MCP servers: postgres, filesystem, memory, membros
- ☐ Webhook WhatsApp + sessão de conversa
- ☐ Comandos básicos via WhatsApp
- ☐ Dashboard com KPIs reais
- ☐ Auditoria básica

FASE 2 — Operações Essenciais (4 semanas)

- ☐ Agente Secretaria completo

☐

Agente Escalas + algoritmo de geração

☐

Agente Financeiro (lançamentos + relatórios simples)

☐

Agente Analytics (NL → SQL seguro)

☐

Portal do Membro

☐

Controle de frequência (QR)

☐

Emissão de documentos (PDF)

FASE 3 — Mídia e Comunicação (4 semanas)

☐

Agente Comunicação + Social Media

☐

Agente Designer + Estúdio Criativo

☐

Pipeline de vídeo básico (FFmpeg + Whisper + cortes)

☐

Publicação multi-canal

☐

CMS + Website por tenant

☐

Newsletter e campanhas

FASE 4 — Pastoral e Avançado (3-4 semanas)

☐

Agente Pastoral + detecção de crise

☐

Agente Sermões + RAG completo

☐

Agente RH Ministerial

☐

Agente Eventos (orquestração completa)

☐

Agente Jurídico/LGPD

☐

App Mobile MVP (Expo)

☐

Human-in-the-loop polido

FASE 5 — Escala, Marketplace e Polish (contínuo)

- ☐ Marketplace
 - ☐ API pública documentada
 - ☐ Observabilidade completa
 - ☐ i18n
 - ☐ Testes de carga e hardening
 - ☐ White-label
 - ☐ Documentação para clientes finais
 - ☐ Onboarding guiado por IA
-

PARTE 18 — SCRIPTS DE MCP (LINHA DE COMANDO)

Arquivo `scripts/install-mcps.sh` (expandido):

```
#!/usr/bin/env bash
set -euo pipefail

echo "=== Assembleia IA – Instalação de Model Context Protocol Servers ==="

# Dependências globais
npm install -g @modelcontextprotocol/sdk 2>/dev/null || true

# MCPs oficiais úteis
npx -y @modelcontextprotocol/server-filesystem "$(pwd)"
npx -y @modelcontextprotocol/server-memory
npx -y @modelcontextprotocol/server-sequential-thinking
npx -y @modelcontextprotocol/server-fetch
npx -y @modelcontextprotocol/server-puppeteer

# PostgreSQL (ajuste a connection string)
# npx -y @modelcontextprotocol/server-postgres "$DATABASE_URL"

echo ""
echo "MCPs customizados devem ser desenvolvidos em packages/mcp-servers/"
echo "Estrutura esperada de cada MCP:"
echo "  packages/mcp-servers/<nome>/"
echo "    ├── package.json"
echo "    ├── src/index.ts"
echo "    ├── src/tools/"
echo "    └── README.md"
echo ""
echo "Após build, registre no arquivo de configuração do cliente MCP"
echo "(claude_desktop_config.json, .cursor/mcp.json, antigravity config, etc.)"
echo ""
echo "Lista obrigatória de MCPs a implementar:"
echo "  1. mcp-whatsapp"
echo "  2. mcp-membros"
echo "  3. mcp-financeiro"
echo "  4. mcp-escalas"
echo "  5. mcp-designer"
echo "  6. mcp-video"
echo "  7. mcp-calendar"
echo "  8. mcp-email"
echo "  9. mcp-storage"
echo " 10. mcp-analytics"
echo " 11. mcp-lgpd"
echo " 12. mcp-social"
echo " 13. mcp-eventos"
echo " 14. mcp-sermoes"
echo ""
echo "Cada MCP deve expor tools com nomes estáveis, schemas Zod/JSON Schema"
echo "e documentação de erros."
```

PARTE 19 — REQUISITOS NÃO FUNCIONAIS (EXPANDIDOS)

Categoria	Requisito	Meta
Disponibilidade	Uptime mensal	99.9% (Essencial/Crescer), 99.95% (Impactar/Dominar)
Latência	API p95	< 200 ms (leitura), < 500 ms (escrita)
Latência IA	Comando simples WhatsApp	< 8 s até primeira resposta útil
Throughput	Requisições	5.000+ RPS sustentados no core
Escala	Tenants	10.000+ igrejas
Escala	Usuários concurrent	100.000+
RPO	Perda de dados	≤ 15 minutos
RTO	Recuperação	≤ 1 hora
Segurança	MFA	Obrigatório para admin
Privacidade	LGPD	100% das bases legais mapeadas
Acessibilidade	WCAG	2.2 AA
Performance web	Lighthouse	≥ 90 em Performance e Accessibility
Custo de IA	Controle	Budget por tenant + alertas

PARTE 20 — RISCOS E MITIGAÇÕES

Risco	Impacto	Mitigação
Alucinação de agente em dados financeiros	Alto	Tools somente leitura + aprovação humana + validação de schema
Vazamento cross-tenant	Crítico	RLS + testes automatizados de isolamento + review de código
Banimento Meta (WhatsApp)	Alto	Templates aprovados, opt-in, rate limit, fallback para e-mail/SMS
Custo explosivo de tokens	Médio	Caching de embeddings, modelos menores para tarefas simples, budgets
Resistência de usuários a IA	Médio	Human-in-the-loop generoso no início, transparência, treinamento
Complexidade de vídeo	Médio	Pipeline assíncrono, filas, status claro, fallback manual

PARTE 21 — GLOSSÁRIO

- **Tenant:** uma igreja (ou organização) cliente da plataforma
- **Agente Mestre:** orquestrador central de IA
- **MCP:** Model Context Protocol — padrão de tools para agentes
- **HITL:** Human-in-the-Loop
- **RLS:** Row Level Security do PostgreSQL
- **WABA:** WhatsApp Business Account
- **RAG:** Retrieval-Augmented Generation

PARTE 22 — INSTRUÇÕES FINAIS AO AGENTE DE DESENVOLVIMENTO

1. Leia este documento completo antes de qualquer código.
2. Crie o monorepo exatamente na estrutura da Parte 2.
3. Implemente o schema da Parte 3 com RLS desde o primeiro migration.
4. Faça o auth multi-tenant funcionar de ponta a ponta.
5. Só então avance para o Agente Mestre e WhatsApp.
6. Cada agente deve ter:
 - System prompt versionado
 - Lista de tools (MCPs)
 - Exemplos de input/output
 - Testes de regressão
7. Documente decisões de arquitetura em `/docs/adr/`.
8. Mantenha o README atualizado com como rodar localmente em < 10 minutos.
9. Nunca commit secrets.
10. Ao concluir uma fase, marque os checkboxes e abra PR descritivo.

Você está autorizado e esperado a gerar milhares de linhas de código de alta qualidade, migrations, testes, Dockerfiles, manifests Kubernetes, scripts e documentação.

Comece pela Fase 0 agora.

FIM DA ESPECIFICAÇÃO v2.0.0 — ASSEMBLEIA IA

Este documento substitui a versão 1.0.0 e é a especificação oficial para implementação. # ASSEMBLEIA IA — VOLUME 2: APÊNDICES TÉCNICOS

Complemento obrigatório da Especificação v2.0.0 (Volume 1)

Leia em conjunto com `ASSEMBLEIA_IA_ESPECIFICACAO_COMPLETA.md`.

APÊNDICE A — SCHEMA SQL COMPLETO (DDL)

Traduzir para Prisma/Drizzle. Aplicar RLS em toda tabela com `tenant_id`.

```

CREATE EXTENSION IF NOT EXISTS "uuid-oss";
CREATE EXTENSION IF NOT EXISTS "pgcrypto";
CREATE EXTENSION IF NOT EXISTS "vector";
CREATE EXTENSION IF NOT EXISTS "pg_trgm";

CREATE TYPE tenant_status AS ENUM ('trial', 'active', 'suspended', 'cancelled');
CREATE TYPE person_type AS ENUM ('member', 'visitor', 'leader', 'child', 'volunteer');
CREATE TYPE person_status AS ENUM ('active', 'inactive', 'transferred', 'deceased', 'archived');
CREATE TYPE transaction_type AS ENUM ('income', 'expense', 'transfer');
CREATE TYPE payment_method AS ENUM ('pix', 'cash', 'credit_card', 'debit_card', 'bank_transfer', 'boleto', 'other');
CREATE TYPE event_status AS ENUM ('draft', 'published', 'cancelled', 'completed');
CREATE TYPE scale_assignment_status AS ENUM ('pending', 'confirmed', 'declined', 'replaced');
CREATE TYPE agent_run_status AS ENUM ('pending', 'running', 'waiting_approval', 'completed', 'failed', 'cancelled');
CREATE TYPE consent_purpose AS ENUM ('membership', 'marketing', 'pastoral_care', 'media', 'analytics');

CREATE TABLE tenants (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  slug TEXT NOT NULL UNIQUE,
  custom_domain TEXT UNIQUE,
  plan TEXT NOT NULL DEFAULT 'essencial',
  status tenant_status NOT NULL DEFAULT 'trial',
  settings JSONB NOT NULL DEFAULT '{}',
  branding JSONB NOT NULL DEFAULT '{}',
  whatsapp_phone_number_id TEXT,
  whatsapp_business_account_id TEXT,
  trial_ends_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email TEXT NOT NULL UNIQUE,
  phone TEXT,
  password_hash TEXT,
  full_name TEXT NOT NULL,
  avatar_url TEXT,
  status TEXT NOT NULL DEFAULT 'active',
  mfa_enabled BOOLEAN NOT NULL DEFAULT false,
  mfa_secret TEXT,
  email_verified_at TIMESTAMPTZ,
  last_login_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE tenant_users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  role TEXT NOT NULL,
  permissions JSONB NOT NULL DEFAULT '[]',
  person_id UUID,
  status TEXT NOT NULL DEFAULT 'active',
  invited_at TIMESTAMPTZ,
  joined_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE(tenant_id, user_id)
);

CREATE TABLE people (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
  type person_type NOT NULL DEFAULT 'member',
  status person_status NOT NULL DEFAULT 'active',
  full_name TEXT NOT NULL,
  preferred_name TEXT,
  birth_date DATE,
  gender TEXT,
  marital_status TEXT,
  cpf TEXT,
  rg TEXT,
  email TEXT,
  phones JSONB NOT NULL DEFAULT '[]',
  address JSONB,
  photo_url TEXT,
  baptism_date DATE,
  membership_date DATE,
  transfer_from TEXT,
  transfer_to TEXT,
  tags TEXT[] NOT NULL DEFAULT '{}',
  custom_fields JSONB NOT NULL DEFAULT '{}',
  notes TEXT,
  lgpd_consent_at TIMESTAMPTZ,
  lgpd_consent_version TEXT,
  created_by UUID REFERENCES users(id),
  updated_by UUID REFERENCES users(id),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  deleted_at TIMESTAMPTZ
);

CREATE INDEX idx_people_name_trgm ON people USING gin (full_name gin_trgm_ops);
CREATE INDEX idx_people_tenant_email ON people(tenant_id, email) WHERE email IS NOT NULL;
CREATE INDEX idx_people_tenant_status ON people(tenant_id, status);
CREATE INDEX idx_people_tags ON people USING gin (tags);

CREATE TABLE families (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
  name TEXT NOT NULL,
  head_id UUID REFERENCES people(id),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),

```

```

    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE family_members (
    family_id UUID NOT NULL REFERENCES families(id) ON DELETE CASCADE
    person_id UUID NOT NULL REFERENCES people(id) ON DELETE CASCADE
    relationship TEXT NOT NULL
    PRIMARY KEY (family_id, person_id)
);

CREATE TABLE departments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    name TEXT NOT NULL
    slug TEXT NOT NULL
    parent_id UUID REFERENCES departments(id),
    leader_id UUID REFERENCES people(id),
    description TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE(tenant_id, slug)
);

CREATE TABLE department_members (
    department_id UUID NOT NULL REFERENCES departments(id) ON DELETE CASCADE
    person_id UUID NOT NULL REFERENCES people(id) ON DELETE CASCADE
    role TEXT,
    joined_at DATE
    PRIMARY KEY (department_id, person_id)
);

CREATE TABLE competencies (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    name TEXT NOT NULL
    category TEXT,
    UNIQUE(tenant_id, name)
);

CREATE TABLE person_competencies (
    person_id UUID NOT NULL REFERENCES people(id) ON DELETE CASCADE
    competency_id UUID NOT NULL REFERENCES competencies(id) ON DELETE CASCADE
    level INT DEFAULT 1,
    PRIMARY KEY (person_id, competency_id)
);

CREATE TABLE events (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    title TEXT NOT NULL,
    slug TEXT NOT NULL
    description TEXT,
    type TEXT,
    status event_status NOT NULL DEFAULT 'draft',
    starts_at TIMESTAMPTZ NOT NULL
    ends_at TIMESTAMPTZ,
    location TEXT,
    location_details JSONB,
    capacity INT,
    registration_required BOOLEAN NOT NULL DEFAULT false,
    cover_image_url TEXT,
    budget_id UUID,
    created_by UUID REFERENCES users(id),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE(tenant_id, slug)
);

CREATE TABLE event_registrations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    event_id UUID NOT NULL REFERENCES events(id) ON DELETE CASCADE,
    person_id UUID REFERENCES people(id),
    guest_name TEXT,
    guest_email TEXT,
    guest_phone TEXT,
    status TEXT NOT NULL DEFAULT 'registered',
    checked_in_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE scale_templates (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    department_id UUID REFERENCES departments(id),
    name TEXT NOT NULL
    slots JSONB NOT NULL
);

CREATE TABLE scales (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    department_id UUID REFERENCES departments(id),
    event_id UUID REFERENCES events(id),
    date DATE NOT NULL
    title TEXT,
    status TEXT NOT NULL DEFAULT 'draft',
    published_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE scale_assignments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    scale_id UUID NOT NULL REFERENCES scales(id) ON DELETE CASCADE,
    person_id UUID NOT NULL REFERENCES people(id),
    role TEXT NOT NULL
    status scale_assignment_status NOT NULL DEFAULT 'pending',

```

```

        notified_at TIMESTAMPTZ,
        responded_at TIMESTAMPTZ,
        notes TEXT
    );

CREATE TABLE availability (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    person_id UUID NOT NULL REFERENCES people(id) ON DELETE CASCADE,
    starts_on DATE NOT NULL,
    ends_on DATE NOT NULL,
    is_available BOOLEAN NOT NULL DEFAULT false,
    reason TEXT
);

CREATE TABLE financial_accounts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    name TEXT NOT NULL,
    type TEXT NOT NULL,
    currency TEXT NOT NULL DEFAULT 'BRL',
    initial_balance NUMERIC(15,2) NOT NULL DEFAULT 0,
    current_balance NUMERIC(15,2) NOT NULL DEFAULT 0,
    bank_code TEXT,
    agency TEXT,
    account_number TEXT,
    is_active BOOLEAN NOT NULL DEFAULT true
);

CREATE TABLE cost_centers (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    name TEXT NOT NULL,
    code TEXT,
    parent_id UUID REFERENCES cost_centers(id)
);

CREATE TABLE chart_of_accounts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    code TEXT NOT NULL,
    name TEXT NOT NULL,
    type TEXT NOT NULL,
    parent_id UUID REFERENCES chart_of_accounts(id),
    UNIQUE(tenant_id, code)
);

CREATE TABLE transactions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    type transaction_type NOT NULL,
    amount NUMERIC(15,2) NOT NULL CHECK (amount > 0),
    currency TEXT NOT NULL DEFAULT 'BRL',
    date DATE NOT NULL,
    description TEXT,
    account_id UUID REFERENCES financial_accounts(id),
    category_id UUID REFERENCES chart_of_accounts(id),
    cost_center_id UUID REFERENCES cost_centers(id),
    person_id UUID REFERENCES people(id),
    payment_method payment_method,
    status TEXT NOT NULL DEFAULT 'confirmed',
    reconciled BOOLEAN NOT NULL DEFAULT false,
    reconciliation_batch_id UUID,
    external_id TEXT,
    receipt_url TEXT,
    metadata JSONB NOT NULL DEFAULT '{}',
    created_by UUID REFERENCES users(id),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_tx_tenant_date ON transactions(tenant_id, date);
CREATE INDEX idx_tx_tenant_person ON transactions(tenant_id, person_id);

CREATE TABLE budgets (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    name TEXT NOT NULL,
    year INT NOT NULL,
    month INT,
    status TEXT NOT NULL DEFAULT 'draft'
);

CREATE TABLE budget_lines (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    budget_id UUID NOT NULL REFERENCES budgets(id) ON DELETE CASCADE,
    category_id UUID REFERENCES chart_of_accounts(id),
    cost_center_id UUID REFERENCES cost_centers(id),
    planned_amount NUMERIC(15,2) NOT NULL,
    notes TEXT
);

CREATE TABLE assets (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    name TEXT NOT NULL,
    category TEXT,
    serial_number TEXT,
    purchase_date DATE,
    purchase_value NUMERIC(15,2),
    current_value NUMERIC(15,2),
    location TEXT,
    status TEXT NOT NULL DEFAULT 'active',
    qr_code TEXT UNIQUE,
    warranty_until DATE,

```

```

photo_url TEXT,
notes TEXT,
created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE sermon_series (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
  title TEXT NOT NULL,
  description TEXT,
  cover_url TEXT
);

CREATE TABLE sermons (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
  series_id UUID REFERENCES sermon_series(id),
  title TEXT NOT NULL,
  preacher_id UUID REFERENCES people(id),
  preached_at TIMESTAMPTZ,
  audio_url TEXT,
  video_url TEXT,
  transcript TEXT,
  summary TEXT,
  verses JSONB NOT NULL DEFAULT '{}',
  tags TEXT[] NOT NULL DEFAULT '{}',
  embedding vector(1536),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_sermons_embedding ON sermons USING hnsw (embedding vector_cosine_ops);

CREATE TABLE conversations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
  channel TEXT NOT NULL,
  external_id TEXT,
  person_id UUID REFERENCES people(id),
  user_id UUID REFERENCES users(id),
  status TEXT NOT NULL DEFAULT 'open',
  last_message_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE conversation_messages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  conversation_id UUID NOT NULL REFERENCES conversations(id) ON DELETE CASCADE,
  direction TEXT NOT NULL,
  content_type TEXT NOT NULL DEFAULT 'text',
  content TEXT,
  media_url TEXT,
  agent_run_id UUID,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE agent_runs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
  agent_name TEXT NOT NULL,
  user_id UUID REFERENCES users(id),
  person_id UUID REFERENCES people(id),
  channel TEXT,
  input JSONB NOT NULL,
  output JSONB,
  status agent_run_status NOT NULL DEFAULT 'pending',
  tokens_input INT,
  tokens_output INT,
  cost_estimate NUMERIC(10,6),
  error TEXT,
  requires_approval BOOLEAN NOT NULL DEFAULT false,
  approved_by UUID REFERENCES users(id),
  approved_at TIMESTAMPTZ,
  started_at TIMESTAMPTZ,
  finished_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE approval_requests (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
  agent_run_id UUID REFERENCES agent_runs(id),
  action_type TEXT NOT NULL,
  payload JSONB NOT NULL,
  status TEXT NOT NULL DEFAULT 'pending',
  requested_to UUID[] NOT NULL,
  resolved_by UUID REFERENCES users(id),
  resolved_at TIMESTAMPTZ,
  expires_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE agent_memories (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
  user_id UUID REFERENCES users(id),
  person_id UUID REFERENCES people(id),
  content TEXT NOT NULL,
  metadata JSONB NOT NULL DEFAULT '{}',
  embedding vector(1536),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_memories_embedding ON agent_memories USING hnsw (embedding vector_cosine_ops);

CREATE TABLE sites (

```

```

    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE UNIQUE
    domain TEXT,
    theme JSONB NOT NULL DEFAULT '{}',
    settings JSONB NOT NULL DEFAULT '{}',
    published BOOLEAN NOT NULL DEFAULT false
};

CREATE TABLE pages (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    site_id UUID NOT NULL REFERENCES sites(id) ON DELETE CASCADE,
    slug TEXT NOT NULL,
    title TEXT NOT NULL,
    content JSONB NOT NULL DEFAULT '{}',
    status TEXT NOT NULL DEFAULT 'draft',
    seo JSONB NOT NULL DEFAULT '{}',
    published_at TIMESTAMPTZ,
    UNIQUE(site_id, slug)
);

CREATE TABLE audit_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID,
    actor_user_id UUID,
    actor_type TEXT,
    action TEXT NOT NULL,
    resource_type TEXT,
    resource_id UUID,
    old_values JSONB,
    new_values JSONB,
    ip INET,
    user_agent TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_audit_tenant_created ON audit_logs(tenant_id, created_at DESC);

CREATE TABLE consent_records (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    person_id UUID NOT NULL REFERENCES people(id) ON DELETE CASCADE,
    purpose consent_purpose NOT NULL,
    granted BOOLEAN NOT NULL,
    version TEXT NOT NULL,
    granted_at TIMESTAMPTZ,
    revoked_at TIMESTAMPTZ,
    evidence JSONB,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE attendance_records (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    event_id UUID REFERENCES events(id),
    person_id UUID NOT NULL REFERENCES people(id),
    check_in_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    method TEXT NOT NULL DEFAULT 'manual',
    UNIQUE(event_id, person_id)
);

ALTER TABLE people ENABLE ROW LEVEL SECURITY;
CREATE POLICY people_tenant_isolation ON people
    USING (tenant_id = NULLIF(current_setting('app.current_tenant_id', true), '')::uuid);

CREATE OR REPLACE FUNCTION set_current_tenant(p_tenant_id UUID) RETURNS void AS $$
BEGIN
    PERFORM set_config('app.current_tenant_id', p_tenant_id::text, true);
END
$$ LANGUAGE plpgsql;

```

Repetir RLS em todas as tabelas com `tenant_id`. NestJS interceptor chama `set_current_tenant` após validar JWT.

APÊNDICE B — ENDPOINTS REST (CATÁLOGO)

Base `/api/v1`. Bearer JWT. Tenant via claim ou subdomain.

Auth: POST register, login, refresh, logout, mfa/enable, mfa/verify, forgot-password, reset-password | GET me

Tenant: GET/PATCH current, PATCH branding, GET users, POST users/invite, PATCH/DELETE users/:id

People: GET (filtros), POST, GET/:id, PATCH/:id, DELETE/:id, POST import, GET/:id/history, POST/:id/photo

Families, Departments, Competencies: CRUD + vínculos de membros e competências

Events: CRUD + publish, registrations, check-in

Scales: GET, POST generate, POST publish, POST assignments respond, GET my

Finance: accounts, transactions (CRUD/bulk), reconcile, reports (summary, cashflow, generate), receipts

Assets: CRUD + movements + inventory-scan

Sermons/Media: CRUD, transcribe, search semântico, library

Agents: POST run, GET runs, GET runs/:id, GET approvals, POST approvals/:id/resolve

Webhooks: whatsapp (GET verify + POST), payments/:provider

CMS: sites/current, pages CRUD, publish

Mobile: me/card, me/scale, me/prayer-requests, me/contributions

APÊNDICE C — USER STORIES ESSENCIAIS

1. **Visitante via WhatsApp** — secretária cadastra por mensagem; extrai dados; confirma; cria visitor.
2. **Escala automática** — líder pede geração; sistema equilibra carga; notifica; confirma; publica.
3. **Prestação de contas** — tesoureiro solicita período; PDF + gráficos + vs orçamento.
4. **Alerta de ausência** — 4 semanas sem presença → alerta pastor + agente pastoral opcional.
5. **Shorts de culto** — upload → 10 shorts + legendas → aprovação → Instagram/YouTube.
6. **Pedido de oração** — membro no app; equipe notificada; status acompanhável.
7. **PIX dízimo** — gera QR; webhook confirma; transaction + recibo WhatsApp.
8. **Crise pastoral** — detecção de risco; empatia sem métodos; encaminha pastor; prioridade crise.
9. **Busca de sermão** — “perdão últimos 2 anos”; busca híbrida; trechos + links.

10. **Onboarding igreja** — wizard branding + WhatsApp + plano de contas; operacional < 30 min.

Expandir para 100+ no backlog durante implementação.

APÊNDICE D — SYSTEM PROMPTS

Mestre: Orquestra. Entende intenção. Delega. Nunca inventa dados. HITL para ações críticas. Tom pastoral pt-BR. Resume ao final.

Pastoral: Acolhe e tria. Crise → empatia, sem métodos, encaminha pastor, notifica. Confidencial.

Financeiro: Precisão total. Threshold de aprovação. Só com permissão. Alerta anomalias.

Designer: Aplica branding obrigatório (logo, cores, fontes). Pergunta formato se faltar.

Escalas: Justiça e equilíbrio. Draft → notify → confirm → publish.

Vídeo: Whisper → cortes → shorts → thumbs → publish multi-canal com aprovação configurável.

Analytics: NL → SQL seguro (SELECT + tenant_id forçado). Sem mutação.

Jurídico/LGPD: Consentimentos, direitos do titular, retenção legal.

APÊNDICE E — STATE MACHINES

- **Escala:** draft → generating → proposed → notifying → partially_confirmed → confirmed → published → archived
 - **Evento:** draft → published → registration_open → registration_closed → in_progress → completed
 - **Agent Run:** pending → running → waiting_approval → completed | failed | cancelled
 - **Transaction:** pending → confirmed → reconciled | cancelled | refunded
-

APÊNDICE F — ERROS PADRÃO

AUTH_001 401 | AUTH_002 403 | TENANT_001 404 | TENANT_002 403 |
VALIDATION_001 422 | FINANCE_001 402 | AGENT_001 429 |
AGENT_002 503 | WHATSAPP_001 502 | CONFLICT_001 409

Body: { "error": { "code", "message", "details", "request_id" } }

APÊNDICE G — GO-LIVE CHECKLIST

Branding | Admins + MFA | WhatsApp + templates | Contas + plano de
contas | Departamentos | Evento teste | Privacidade/termos |
Consentimentos LGPD | Backup/monitoramento | Treinamento

APÊNDICE H — MODELOS IA

Orquestração: GPT-5.5 / Claude 4
SQL: modelo forte + validador
Embeddings: text-embedding-3-large
Imagem: Flux / Ideogram
Transcrição: Whisper large-v3
TTS: ElevenLabs / OpenAI
Visão: GPT-4o / Gemini
BYOK nos planos superiores.

APÊNDICE I — CUSTO IA

Registrar por run: tokens, model, cost_usd, latency. Agregar por tenant/
dia. Budgets e alertas.

APÊNDICE J — ESCALA

Particionar logs/mensagens | Read replicas | Redis cache | CDN mídia |
Workers vídeo auto-scale | PgBouncer | Rate limit por tenant
Meta: API p95 < 200ms | WhatsApp simples < 8s

APÊNDICE K — install-mcps.sh

```
#!/usr/bin/env bash
set -euo pipefail
npx -y @modelcontextprotocol/server-filesystem "$(pwd)"
npx -y @modelcontextprotocol/server-memory
npx -y @modelcontextprotocol/server-sequential-thinking
npx -y @modelcontextprotocol/server-fetch
npx -y @modelcontextprotocol/server-puppeteer
echo "Custom MCPs em packages/mcp-servers/: whatsapp membros financeiro escalas designer video calendar email storage analytics lgpd
social eventos sermoes"
```

APÊNDICE L — FASES (LEMBRETE)

0 Fundação → 1 Core+Mestre+WhatsApp → 2 Operações → 3 Mídia → 4 Pastoral → 5 Marketplace/Escala

Cada fase: testes OK, README, demo, segurança.

FIM VOLUME 2 # ASSEMBLEIA IA — VOLUME 3 ## ESTRUTURA ORGANIZACIONAL DA ASSEMBLEIA DE DEUS + MODELO DE DADOS COMPLETO PARA CARGOS E MEMBROS

Versão: 3.0.0

Obrigatório: Este volume especializa a plataforma para a realidade da **Assembleia de Deus no Brasil** (CGADB, CADB, CONAMAD/ Madureira e ministérios autônomos).

Uso: O agente de desenvolvimento deve implementar seed de cargos, departamentos e vínculos membro↔cargo exatamente conforme este documento. Cargos são configuráveis por tenant, mas o catálogo padrão AD deve vir pré-carregado.

1. CONTEXTO DENOMINACIONAL

As Assembleias de Deus no Brasil não formam uma única igreja centralizada. São milhares de igrejas locais e ministérios, agrupados principalmente em:

Convenção / Ramo	Observação
CGADB — Convenção Geral das Assembleias de Deus no Brasil	“Missão” (herança sueca)
CADB — Convenção da Assembleia de Deus no Brasil	Também ligada à tradição Missão
CONAMAD — Convenção Nacional (Ministério Madureira)	Estrutura com episcopado (Bispo Primaz)
Ministérios e convenções estaduais/regionais autônomas	Ex.: Ministério Belém, Ministério do Belém, etc.

Implicação para o software:

Cada **tenant** = uma igreja local (templo sede + congregações) **ou** um ministério completo.

O sistema **não impõe** uma convenção. Oferece catálogo padrão de cargos e departamentos AD, 100% customizável por tenant.

2. HIERARQUIA ESTRUTURAL TÍPICA (IGREJA LOCAL AD)

```
MINISTÉRIO / CONVENÇÃO (opcional, nível acima)
├── IGREJA SEDE (Templo Central)
│   ├── Pastor Presidente / Pastor Titular
│   ├── Pastores Auxiliares / Copastores
│   ├── Corpo de Obreiros (Presbíteros, Evangelistas, Diáconos...)
│   ├── Departamentos (Mocidade, Círculo de Oração, EBD...)
│   ├── Congregações / Pontos de Pregação
│   │   └── Dirigente de Congregação (+ obreiros locais)
│   └── Campos Missionários / Obras Sociais
```

Níveis territoriais suportados no sistema

Nível	Código sugerido	Exemplo
Ministério / Convenção	ministry	Ministério Assembleia de Deus de [Cidade]
Igreja Sede	headquarters	Templo Central
Congregação	congregation	Congregação do Bairro X
Ponto de Pregação / Campo	preaching_point	Ponto de Pregação Rua Y
Célula / Grupo pequeno	cell	Célula da Família Z

Tabela `locations` (ou `church_units`) com `parent_id` para árvore.

3. CATÁLOGO COMPLETO DE CARGOS E FUNÇÕES DA ASSEMBLEIA DE DEUS

Todos os cargos abaixo devem existir no **seed padrão**.

Cada cargo tem: - `code` (único por tenant) - `name` - `category` (eclesiástico | administrativo | departamento | louvor | ensino | apoio | missão) - `rank_order` (para ordenação hierárquica visual) - `requires_consecration` (boolean — se exige consagração formal) - `is_system` (boolean — cargos do seed não apagáveis, só desativáveis) - `description` - `permissions_hint` (sugestão de permissões no sistema)

3.1 Cargos Eclesiásticos / Ministeriais (consagração)

Ordem	Código	Nome	Descrição resumida	Consagração
10	pastor_presidente	Pastor Presidente / Pastor Titular	Líder máximo da igreja local ou ministério. Representante legal em muitos casos.	Sim
20	pastor_auxiliar		Auxilia o pastor presidente na pregação, visitação e governo.	Sim

Ordem	Código	Nome	Descrição resumida	Consagração
		Pastor Auxiliar / Co-pastor		
30	pastor	Pastor	Ministro consagrado; pode pastorear congregação ou área.	Sim
40	evangelista	Evangelista	Foco em evangelização, campanhas e plantio.	Sim
50	missionario	Missionário / Missionária	Enviado a campo missionário (nacional ou transcultural).	Sim
60	presbitero	Presbítero	Ancião; maturidade espiritual; auxilia no governo e disciplina.	Sim
70	diacono	Diácono	Serviço, assistência, organização do culto, mesa da santa ceia.	Sim
80	diaconisa	Diaconisa	Equivalente feminino do diaconato (quando a igreja reconhece).	Sim
90	obreiro	Obreiro / Obreira	Cooperador consagrado ou reconhecido; base do corpo de obreiros.	Sim/Não*
100	cooperador	Cooperador / Auxiliar	Em formação; auxilia antes da consagração plena.	Não
110	aspirante_diaconato	Aspirante ao Diaconato	Em preparação e observação para consagração.	Não

*Em muitas ADs a progressão é: **Cooperador → Diácono → Presbítero → Evangelista → Pastor.**

Observações por convenção

- **Madureira (CONAMAD):** pode haver **Bispo, Bispo Primaz**, estrutura episcopal.
- Incluir no catálogo opcional:

Ordem	Código	Nome
5	bispo_primaz	Bispo Primaz
8	bispo	Bispo
15	pastor_conselheiro	Pastor Conselheiro

3.2 Cargos de Direção de Unidade

Código	Nome	Descrição
dirigente_congregacao	Dirigente de Congregação	Responsável por uma congregação sob a sede
dirigente_ponto	Dirigente de Ponto de Pregação	Responsável por ponto de pregação / campo
dirigente_setor	Dirigente de Setor / Região	Coordena várias congregações (ministérios grandes)
superintendente	Superintendente	Coordenação de área (EBD, missões, etc.)

3.3 Cargos Administrativos da Igreja Local

Código	Nome	Descrição
secretario_geral	Secretário(a) Geral	Atas, documentos, correspondências oficiais
secretario_adjunto	Secretário(a) Adjunto(a)	Auxilia a secretaria
tesoureiro	Tesoureiro(a)	Finanças, dízimos, prestações de contas
tesoureiro_adjunto	Tesoureiro(a) Adjunto(a)	Auxilia a tesouraria
secretario_tesoureiro	Secretário-Tesoureiro	Função combinada (igrejas menores)
conselho_fiscal		Fiscalização financeira

Código	Nome	Descrição
	Membro do Conselho Fiscal	
conselho_administrativo	Membro do Conselho Administrativo / Deliberativo	Governança

3.4 Escola Bíblica Dominical (EBD)

Código	Nome
superintendente_ebd	Superintendente de EBD
secretario_ebd	Secretário(a) de EBD
tesoureiro_ebd	Tesoureiro(a) de EBD
professor_ebd	Professor(a) de EBD
professor_ebd_infantil	Professor(a) EBD Infantil
professor_ebd_jovens	Professor(a) EBD Jovens
professor_ebd_adultos	Professor(a) EBD Adultos
auxiliar_ebd	Auxiliar de EBD

3.5 Círculo de Oração / Senhoras / Mulheres

Código	Nome
dirigente_circulo_oracao	Dirigente do Círculo de Oração
vice_dirigente_circulo	Vice-Dirigente do Círculo de Oração
secretaria_circulo	Secretária do Círculo de Oração
tesoureira_circulo	Tesoureira do Círculo de Oração
dirigente_senhoras	Dirigente de Senhoras / Departamento de Mulheres
intercessora	Intercessora

3.6 Departamento de Varões / Homens

Código	Nome
dirigente_varoes	Dirigente de Varões
vice_dirigente_varoes	Vice-Dirigente de Varões
secretario_varoes	Secretário de Varões

3.7 Mocidade / Jovens / Adolescentes / Crianças

Código	Nome
dirigente_mocidade	Dirigente de Mocidade / Jovens
vice_dirigente_mocidade	Vice-Dirigente de Mocidade
secretario_mocidade	Secretário de Mocidade
dirigente_adolescentes	Dirigente de Adolescentes
dirigente_juvenis	Dirigente de Juvenis
dirigente_infantil	Dirigente do Departamento Infantil
dirigente_crianças	Dirigente de Crianças
auxiliar_infantil	Auxiliar Infantil / Tia(o) do Infantil
dirigente_teens	Dirigente de Teens (quando houver)
dirigente_universitarios	Dirigente de Universitários / Profissionais

3.8 Louvor, Música e Artes

Código	Nome
ministro_louvor	Ministro de Louvor / Regente
dirigente_louvor	Dirigente de Louvor
regente_coral	Regente de Coral
maestro	Maestro / Regente de Orquestra ou Banda
musico	Músico / Instrumentista
vocalista	Vocalista / Cantor(a)
back_vocal	Back Vocal
tecnico_som	Técnico de Som
tecnico_iluminacao	Técnico de Iluminação
dirigente_danca	Dirigente de Dança / Expressão Corporal (quando houver)
dirigente_teatro	Dirigente de Teatro / Drama

3.9 Mídia, Comunicação e Tecnologia

Código	Nome
dirigente_midia	Dirigente de Mídia / Comunicação
operador_projecao	Operador de Projeção / Multimídia
operador_camera	Operador de Câmera
editor_video	Editor de Vídeo
designer_grafico	Designer / Arte
social_media	Responsável por Redes Sociais
fotografo	Fotógrafo(a)

3.10 Recepção, Ordem e Apoio ao Culto

Código	Nome
dirigente_recepcao	Dirigente de Recepção / Boas-Vindas
repcionista	Recepcionista / Porteiro(a) de Acolhimento
porteiro	Porteiro(a)
dirigente_ordem	Dirigente de Ordem / Disciplina no culto
auxiliar_ordem	Auxiliar de Ordem
dirigente_estacionamento	Dirigente de Estacionamento
auxiliar_estacionamento	Auxiliar de Estacionamento
dirigente_limpeza	Dirigente de Limpeza / Zeladoria
zelador	Zelador(a)

3.11 Intercessão, Visitação e Ação Social

Código	Nome
dirigente_intercessao	Dirigente de Intercessão
intercessor	Intercessor(a)
dirigente_visitacao	Dirigente de Visitação
visitador	Visitador(a)
dirigente_acao_social	Dirigente de Ação Social / Assistência
voluntario_social	Voluntário(a) de Ação Social
dirigente_missoes	Dirigente de Missões
dirigente_evangelismo	Dirigente de Evangelismo

3.12 Ensino, Discipulado e Cursos

Código	Nome
dirigente_discipulado	Dirigente de Discipulado
discipulador	Discipulador(a)
dirigente_cursos	Dirigente de Cursos / Escola de Líderes
professor_cursos	Professor(a) de Cursos Bíblicos
conselheiro	Conselheiro(a) Pastoral (leigo treinado)

3.13 Outros cargos frequentes

Código	Nome
dirigente_casais	Dirigente de Ministério de Casais
dirigente_familias	Dirigente de Famílias
dirigente_novo_convertido	Dirigente de Novos Convertidos / Classe de Novos
dirigente_batismo	Coordenador de Preparação para Batismo
dirigente_santa_ceia	Coordenador da Mesa da Santa Ceia
cerimonialista	Cerimonialista (casamentos, eventos)
dirigente_eventos	Dirigente de Eventos
voluntario_geral	Voluntário(a) Geral

4. DEPARTAMENTOS PADRÃO (SEED AD)

Cada departamento pode ter dirigente, vice, secretário, tesoureiro e membros.

Código	Nome do Departamento
ebd	Escola Bíblica Dominical
circulo_oracao	Círculo de Oração
senhoras	Departamento de Senhoras / Mulheres
varoes	Departamento de Varões
mocidade	Mocidade / Jovens
adolescentes	Adolescentes
juvenis	Juvenis
infantil	Departamento Infantil / Crianças
louvor	Ministério de Louvor
coral	Coral
orquestra_banda	Orquestra / Banda
midia	Mídia e Comunicação
recepcao	Recepção e Boas-Vindas
intercessao	Intercessão
visitacao	Visitação
acao_social	Ação Social
missoes	Missões
evangelismo	Evangelismo
discipulado	Discipulado
casais	Ministério de Casais
familias	Ministério de Famílias
novos_convertidos	Novos Convertidos
estacionamento	Estacionamento
limpeza	Limpeza / Zeladoria

Código	Nome do Departamento
ordem	Ordem e Disciplina
eventos	Eventos
secretaria	Secretaria
tesouraria	Tesouraria

Departamentos são hierárquicos (`parent_id`) e vinculados a `location` (sede ou congregação).

5. MODELO DE DADOS — CARGOS, VÍNCULOS E HISTÓRICO

5.1 Tabelas novas / expandidas

```
-- Catálogo de cargos (por tenant, com seed do sistema)
CREATE TABLE positions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
  code TEXT NOT NULL,
  name TEXT NOT NULL,
  category TEXT NOT NULL -- eclesiastico, administrativo, departamento, louvor, ensino, apoio, missao
  rank_order INT NOT NULL DEFAULT 500,
  requires_consecration BOOLEAN NOT NULL DEFAULT false,
  is_system BOOLEAN NOT NULL DEFAULT false -- seed AD
  is_active BOOLEAN NOT NULL DEFAULT true
  description TEXT,
  permissions_hint JSONB NOT NULL DEFAULT '[]',
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE(tenant_id, code)
);

-- Unidades da igreja (sede, congregações, pontos, células)
CREATE TABLE church_units (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
  parent_id UUID REFERENCES church_units(id),
  type TEXT NOT NULL -- headquarters, congregation, preaching_point, cell, sector
  name TEXT NOT NULL,
  code TEXT,
  address JSONB,
  leader_person_id UUID REFERENCES people(id),
  is_active BOOLEAN NOT NULL DEFAULT true,
  opened_at DATE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE(tenant_id, code)
);

-- Departamentos (já existia; expandir)
-- departments: id, tenant_id, church_unit_id, code, name, parent_id, leader_id, ...

-- Vínculo pessoa ↔ cargo (histórico completo)
CREATE TABLE person_positions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
  person_id UUID NOT NULL REFERENCES people(id) ON DELETE CASCADE,
  position_id UUID NOT NULL REFERENCES positions(id),
  church_unit_id UUID REFERENCES church_units(id), -- onde exerce
  department_id UUID REFERENCES departments(id), -- se cargo de departamento
  status TEXT NOT NULL DEFAULT 'active', -- active, suspended, concluded, cancelled
  is_primary BOOLEAN NOT NULL DEFAULT false -- cargo principal exibido no perfil
  started_at DATE NOT NULL
  ended_at DATE,
  consecrated_at DATE -- data da consagração (se houver)
  consecrated_by UUID REFERENCES people(id), -- quem consagrou
  document_url TEXT, -- ata / certificado
  notes TEXT,
  appointed_by UUID REFERENCES users(id),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_person_positions_person ON person_positions(tenant_id, person_id);
CREATE INDEX idx_person_positions_active ON person_positions(tenant_id, status) WHERE status = 'active';

-- Credenciais / carteirinhas ministeriais
CREATE TABLE ministerial_credentials (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
  person_id UUID NOT NULL REFERENCES people(id) ON DELETE CASCADE,
  position_id UUID REFERENCES positions(id),
  credential_number TEXT,
  issued_at DATE,
  expires_at DATE,
  issuing_body TEXT, -- ex: "CGADB", "Convenção Estadual", nome do ministério
  document_url TEXT,
  status TEXT NOT NULL DEFAULT 'valid', -- valid, expired, revoked
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

5.2 Campos adicionais em `people` (membro)

Além dos campos já definidos no Volume 2:

```

membership_number    -- número de matrícula / rol de membros
conversion_date       -- data de conversão
holy_spirit_baptism_date -- batismo com Espírito Santo (doutrina AD)
origin_church         -- igreja de onde veio (transferência)
transfer_letter_url   -- carta de mudança
member_since         -- data de admissão no rol
member_status_detail -- active | under_discipline | leave | transferred_out | deceased
preferred_church_unit_id -- congregação que frequenta
is_tither            -- dizimista (flag informativa)
spiritual_gifts       -- text[] ou relação
availability_notes    -- observações para escala
emergency_contact     -- jsonb

```

5.3 Regras de negócio do vínculo cargo

1. Uma pessoa pode ter **vários cargos** ao mesmo tempo (ex.: Diácono + Dirigente de Louvor + Professor EBD).
2. Apenas **um** `is_primary = true` por pessoa (cargo exibido em destaque).
3. Cargos com `requires_consecration = true` devem registrar `consecrated_at` (ou fluxo de aprovação).
4. Ao encerrar cargo (`ended_at` + status `concluded`), o histórico permanece para relatórios e carteirinha.
5. Escalas (Agente Escalas) consultam `person_positions` + `person_competencies` + `availability`.
6. Relatórios: “lista de diáconos ativos”, “obreiros da congregação X”, “histórico ministerial do membro Y”.

5.4 Permissões sugeridas por categoria de cargo

Categoria	Permissões típicas no sistema
Pastor Presidente	Quase total no tenant (exceto billing SaaS)
Pastor / Presbítero	people:read/write, pastoral, scales approve, events
Diácono / Obreiro	people:read, scales, attendance, prayer
Tesoureiro	finance:*
Secretário	people:*, documents, events
Dirigente de departamento	scales do seu depto, members do depto
Membro comum	me:*, prayer, contributions, events register
Voluntário de mídia	media:*, publish (com aprovação)

O mapeamento exato fica em `role_permissions` + overrides por `person_positions`.

6. FLUXOS ESPECÍFICOS AD

6.1 Admissão de membro

1. Visitante cadastrado → acompanhamento
2. Conversão / decisão registrada
3. Discipulado / classe de novos convertidos
4. Batismo nas águas (data + oficiante)
5. Batismo com Espírito Santo (opcional, data)
6. Admissão no rol de membros (número de matrícula)

7. Carta de mudança (se transferência)
8. Consentimentos LGPD

6.2 Consagração de obreiro

1. Indicação / aspirante
2. Período de observação
3. Aprovação do pastor / conselho
4. Culto de consagração (data, oficiantes)
5. Registro em `person_positions` + credencial
6. Atualização de permissões

6.3 Transferência (carta de mudança)

1. Solicitação
2. Emissão de carta (PDF com dados e cargos)
3. Status `transferred_out`
4. Igreja destino pode importar via código/QR (futuro inter-tenant)

6.4 Disciplina

1. Registro confidencial (acesso restrito a pastores)
 2. Status `under_discipline`
 3. Acompanhamento e restauração
 4. Auditoria completa
-

7. SEED SQL / TYPESCRIPT (RESUMO PARA O AGENTE)

O agente deve criar:

1. Migration das tabelas `positions`, `church_units`, `person_positions`, `ministerial_credentials`
2. Seed `positions` com **todos** os códigos da seção 3 (`is_system = true`)
3. Seed `departments` com todos os da seção 4
4. Seed de uma `church_unit` tipo `headquarters` na criação do tenant
5. UI:
 - Cadastro de membro com aba **Cargos e Ministérios**
 - Filtros por cargo, departamento, congregação
 - Relatório “Corpo de Obreiros”
 - Carteira digital com cargo principal e credencial
6. Agente Secretaria e Agente RH Ministerial devem conhecer todos esses cargos em linguagem natural
 - Ex.: “Liste os diáconos da congregação Centro”
 - “Consagre João Silva como presbítero em 15/03/2026”

8. EXEMPLOS DE COMANDOS EM LINGUAGEM NATURAL (WHATSAPP / CHAT)

- “Cadastre Maria Oliveira como membro, congregação Jardim, batizada em 10/01/2025”
 - “Quem são os diáconos ativos?”
 - “Promova Carlos a presbítero a partir de hoje”
 - “Escale os músicos e o ministro de louvor para domingo”
 - “Quantos membros temos na mocidade?”
 - “Emita carta de mudança do membro Pedro Santos”
 - “Liste o corpo de obreiros da sede”
 - “Quem é o dirigente do círculo de oração?”
-

9. RELATÓRIOS OBRIGATÓRIOS (AD)

1. Rol de membros (ativos, por congregação)
 2. Corpo de obreiros (por cargo e unidade)
 3. Aniversariantes do mês
 4. Batismos do período
 5. Transferências emitidas/recebidas
 6. Frequência EBD e cultos
 7. Dizimistas (agregado, sem expor valores individuais indevidamente)
 8. Mapa de congregações e dirigentes
 9. Histórico ministerial individual (para credencial / currículo)
-

10. CHECKLIST DE IMPLEMENTAÇÃO DESTES VOLUMES

☐

Tabelas `positions`, `church_units`, `person_positions`, `ministerial_credentials`

☐

Seed completo de cargos AD (seção 3)

☐

Seed de departamentos AD (seção 4)

☐

UI de vínculo membro ↔ cargo com histórico

☐

Filtros e relatórios por cargo

☐

- ☐ Carteirinha digital com cargo
 - ☐ Agente Secretaria/RH entendendo nomes de cargos em pt-BR
 - ☐ Fluxos: admissão, consagração, transferência, disciplina
 - ☐ Permissões sugeridas por categoria
 - ☐ Documentação no README do módulo ministerial
-

FIM DO VOLUME 3 — ASSEMBLEIA DE DEUS

Este volume é parte integrante da especificação Assembleia IA e deve ser implementado integralmente para que a plataforma fale a língua real das igrejas Assembleia de Deus no Brasil.